

# BLACKJACK



Presented by  
**Filipa Delfino**  
**Simão Pereira**

# Introduction and Goal



- **What is the project?** An Artificial Intelligence agent trained to play Blackjack.
- **How was it built?** Instead of traditional Deep Learning approaches (like backpropagation), we used Neuroevolution.
- **Technologies:** Python, Pygame (for the visual interface), and Gymnasium (for the game environment).

# Development Environment

## Stack & Baseline Parameters

Python + Gymnasium (Blackjack environment) & NumPy

- ✂ POP\_SIZE = 50 (Active agent population)
- ✂ GENERATIONS = 100 (GA generation limit)
- ✂ DNA\_SIZE = 81 (Total network connection weights)
- ✂ N\_GAMES = 100 (Games played per individual evaluation)

## Phased Methodology:

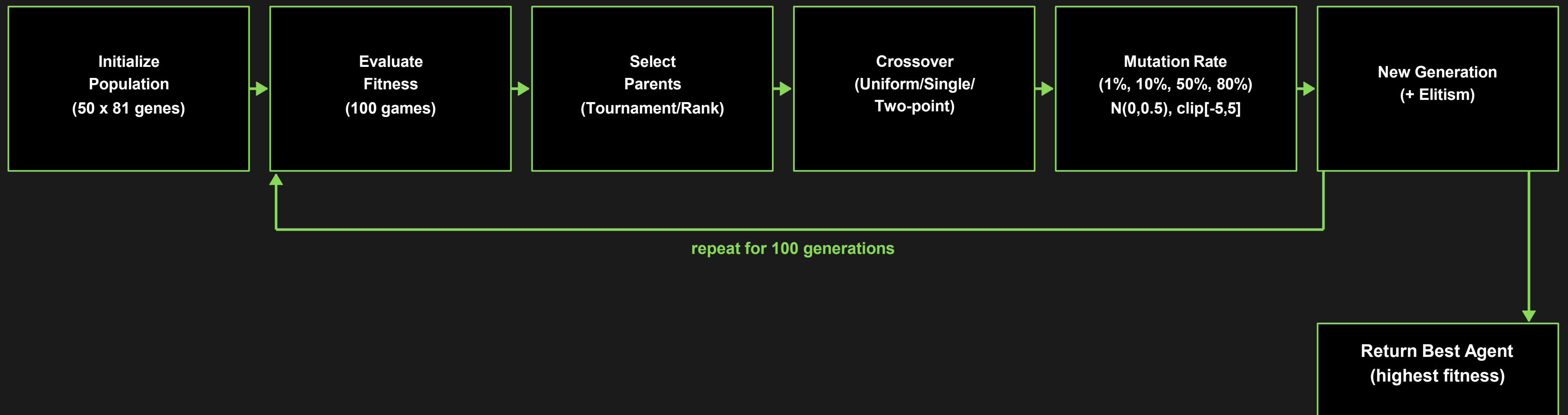
First, the main algorithm runs for 100 generations to validate the environment and the fitness function.

## Systematic Comparison ([analyze.py](#)):

Built upon a validated baseline to rigorously compare selection methods, crossover techniques, and mutation rates

Goal: discover the operator sequence that produces the best agent - the one that wins the most and loses the least (a tie is preferred over a loss).

# SYSTEM FLOWCHART





# PSEUDOCODE

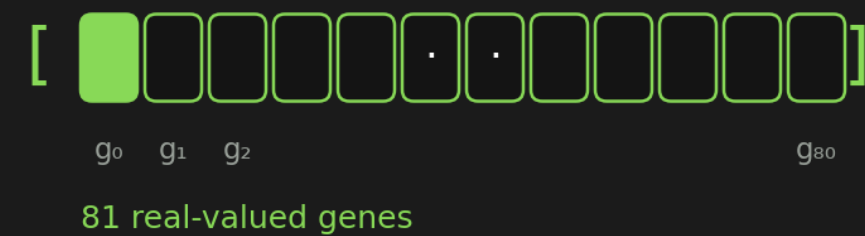
1. Initialize the Blackjack environment and create a population of 50 individuals; each is an 81-gene vector encoding a  $3 \rightarrow 16 \rightarrow 1$  neural network.
2. Repeat for 100 generations:
  - 2.1 Evaluate every individual's fitness: decode it into a neural network and average its reward over N\_GAMES of Blackjack.
  - 2.2 Record the generation's max and average fitness, and keep the best individual seen so far (elitism).
  - 2.3 Refill the population: keep the best individual unchanged, then repeatedly select two parents (Tournament or Rank), recombine them (Uniform, Single-Point or Two-Point crossover) and mutate the offspring at the chosen mutation rate, until the population is full again.
3. Return the best individual found, its fitness, and the max/average fitness history.

# Algorithm

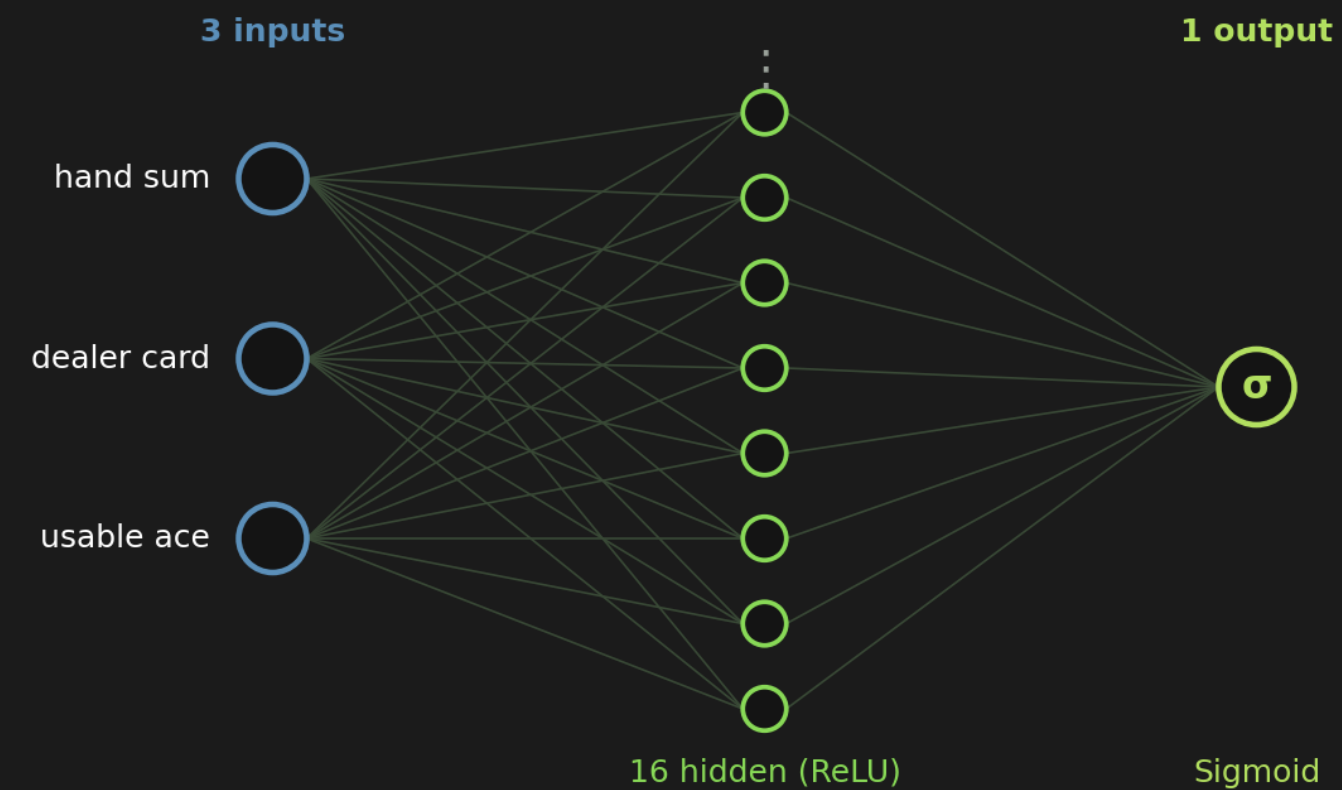
## ① The 81-gene genome

|                 |           |
|-----------------|-----------|
| 3 × 16 weights  | 48        |
| 16 hidden bias  | 16        |
| 16 × 1 weights  | 16        |
| 1 output bias   | 1         |
| <b>DNA_SIZE</b> | <b>81</b> |

One individual = one weight vector



## ② Genome → Network

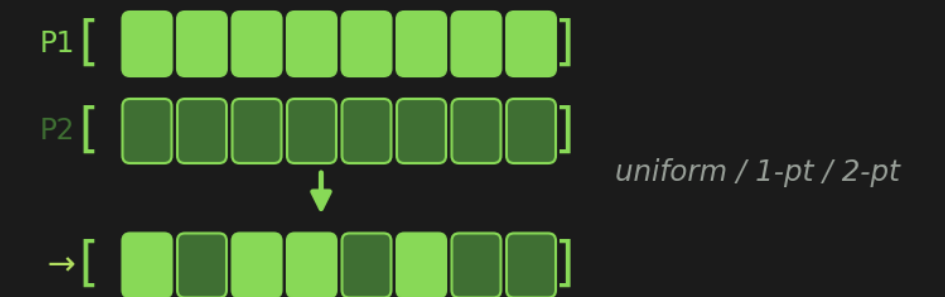


```
if output > 0.5      → HIT
otherwise            → STAND
```

### ③ How it evolves

## Crossover

*select 2 parents → recombine*



## Mutation

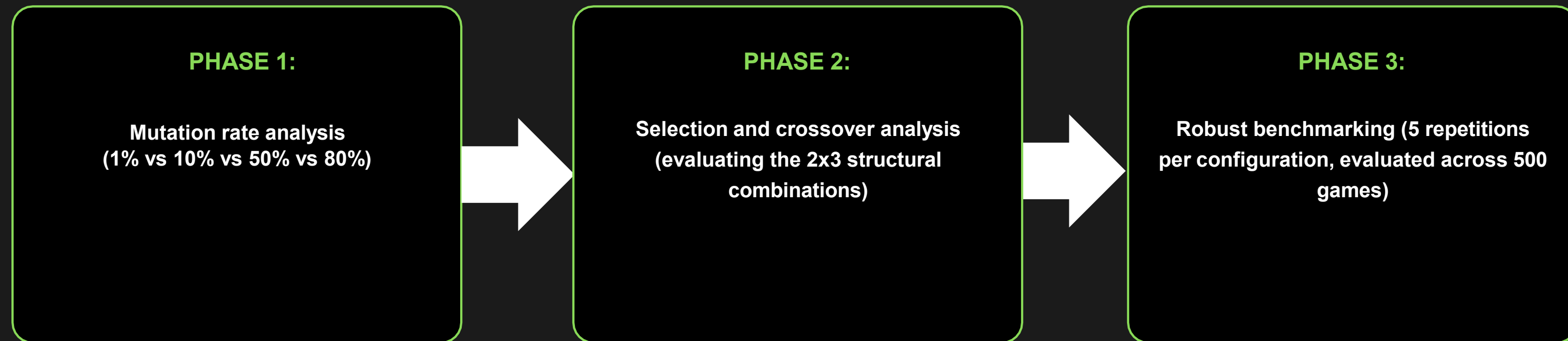
per-gene Gaussian noise



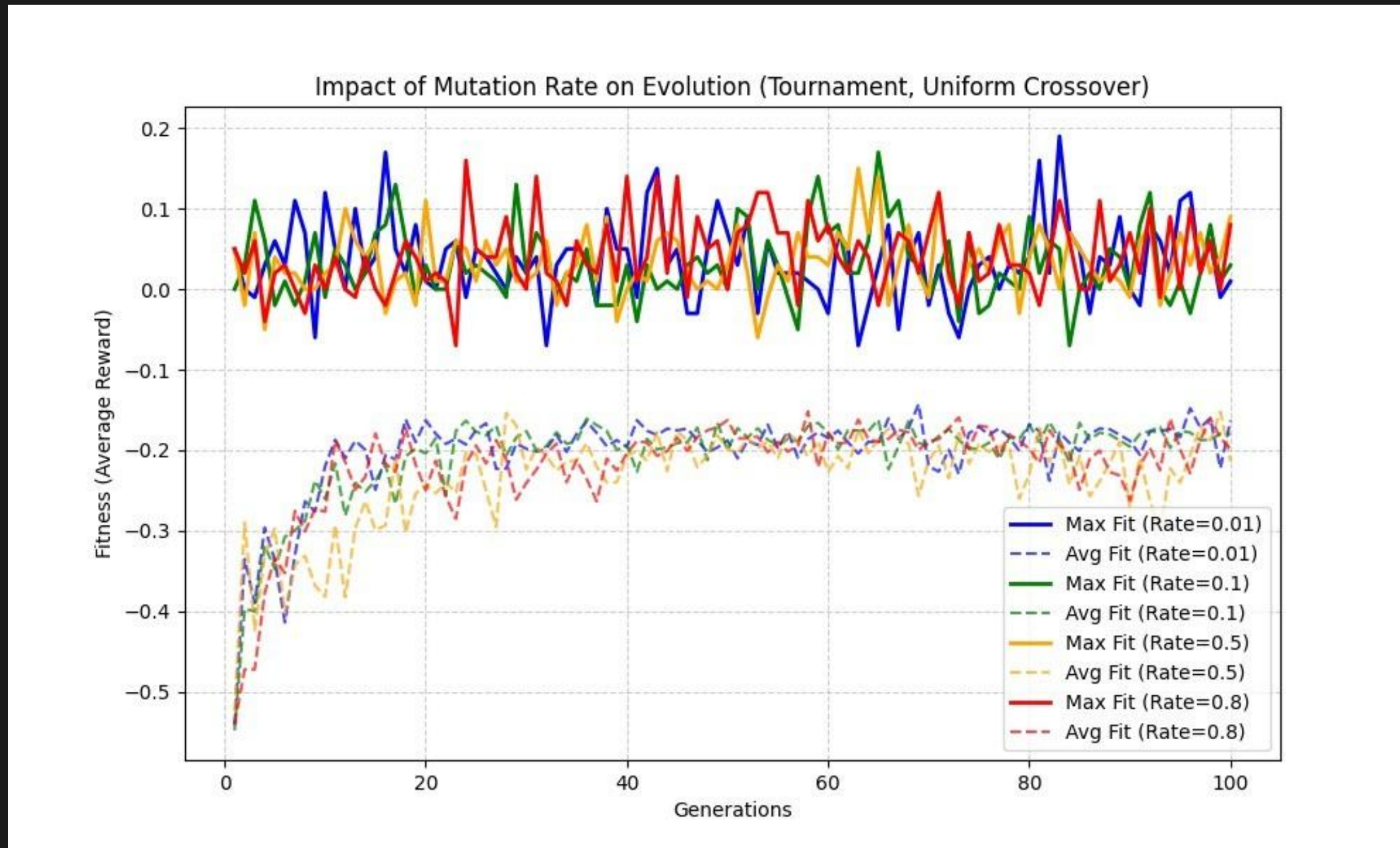
```
gi += N(0, 0.5)  
then clip to [-5, 5]
```

- ★ *best individual kept (elitism)*

# Analysis Methodology



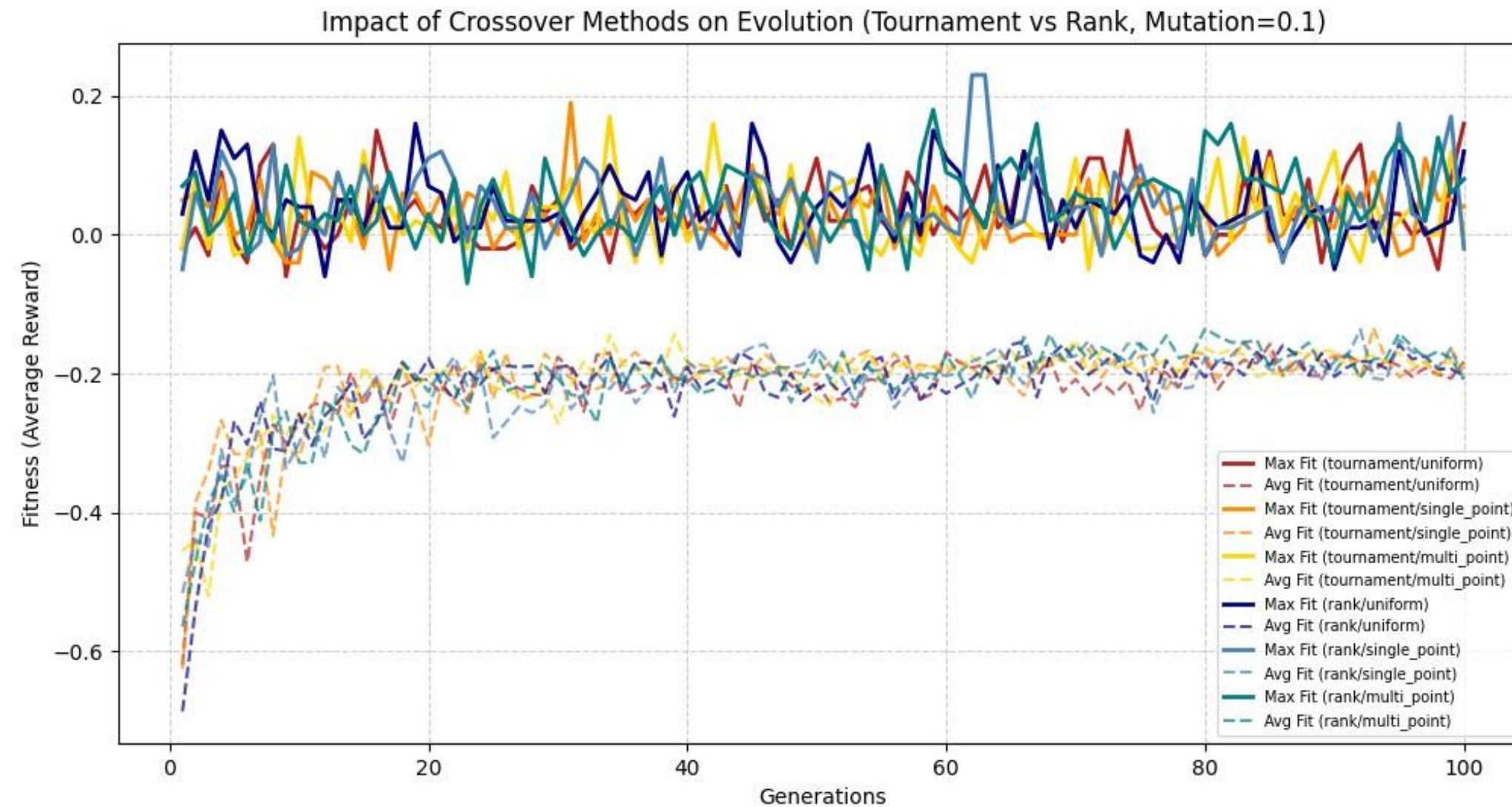
# MUTATION RATE



**Conclusion:** search stability, not final convergence

Rates of 50% and 80% are clearly unstable and converge much more slowly; for this reason, the later analysis phases were simulated only with 1% and 10% mutation rates.



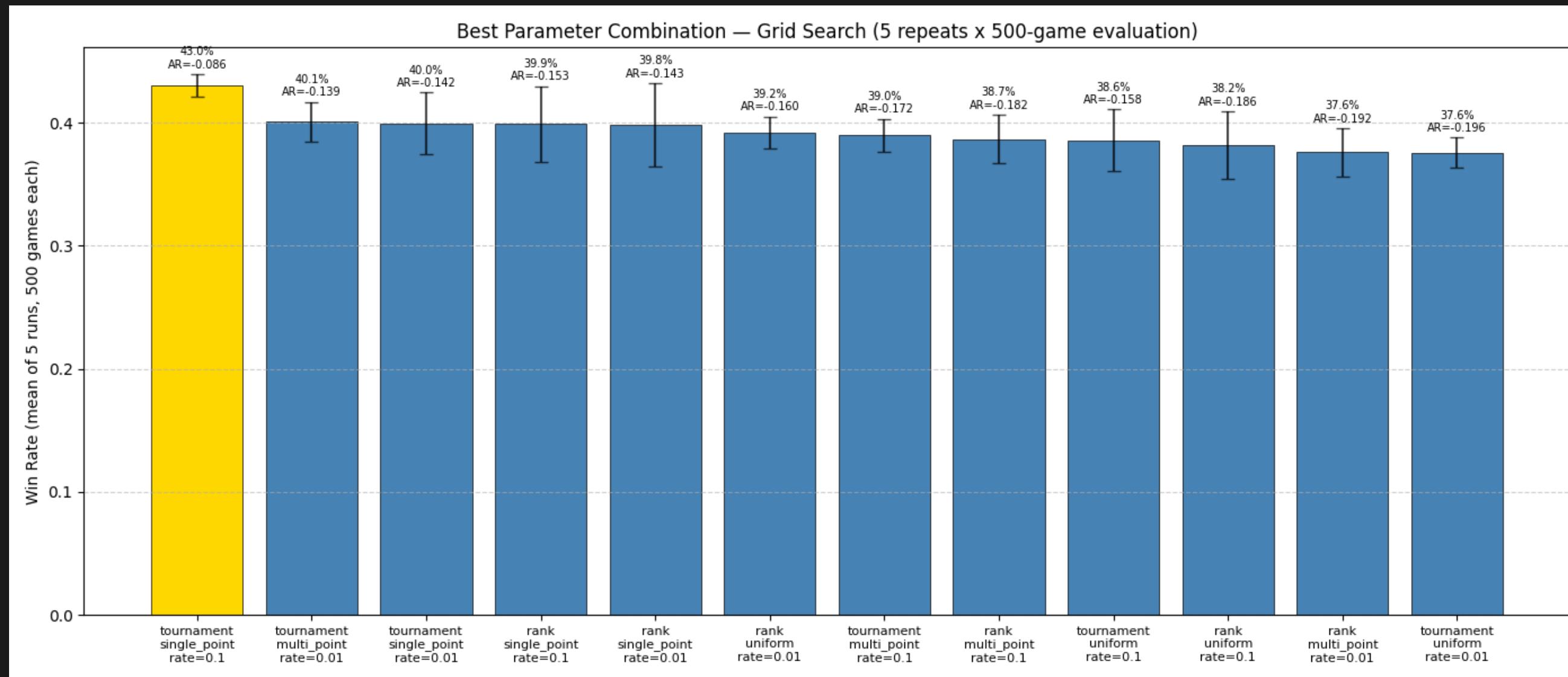


# Results: Selection / Crossover

## Conclusion:

Differences between selection and crossover methods are small; all combinations show overlapping fitness curves.

# Results: Best Combination



## Conclusion:

Win rates mostly overlap within one standard deviation, so most combinations are statistically indistinguishable. Tournament / Single-point / 10% mutation is the exception: across the full grid search (12 combinations x 5 repeats x 500 games) it leads on both metrics at once - highest win rate (43.0%, error bar clear of the rest) and clearly the best average reward (AR=-0.086 vs. -0.139 for the runner-up) - which is enough evidence to call it the most reliable and most "profitable" sequence found.